

Architektury systemów komputerowych: Reference

Data: 2026-06-11

Zakres: 1 - 2

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	s = x ^ y c = x & y	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	s = x ^ y ^ ci co=x&y ci&(x^y)	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B, A B, A\&B$); multipleksowanie wyników brankami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawia $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Zbudowany z dwóch sprzężonych NOR -ów.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R bramkowane AND-em z zegarem. Stan zmienia się tylko przy CLK=1 .
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Bity, bajty i typy danych

1B = 8b, hex = 4b $\Rightarrow$ 1B $\in$ [0x00; 0xFF]. Np. 15213 = 0x3B6D.

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \qquad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \qquad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2). Stąd **B2U** = bity $\rightarrow$ unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (INT_MAX)
TMin	10...0	-2^{w-1} (INT_MIN)
−1	11...1	te same bity co UMax

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu (**< > ==**) $\rightarrow$ **int** promowany do **unsigned** (bity bez zmian, $-1 \rightarrow$ UMax).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (zext)	unsigned : dopisuje 0 z góry
Rozszerz. znak (sext)	signed : powiela bit znaku (MSB)
Obcięcie (trunc)	$u \bmod 2^w$; może zmienić znak
x << k	$x \cdot 2^k$ (sgn./uns.)
u >> k	$\lfloor u/2^k \rfloor$ – logiczne (zera)
x >> k	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	(x + (1<<k)-1) >> k

Strength red.: **x*24** $\rightarrow$ **(x<<5)-(x<<3)**.

5. Operacje, overflow i endianness

UAdd	$(u + v) \bmod 2^w$; carry ignorowane
TAdd	bitowo = UAdd; różne znaki nie przepelniają
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)
Nadmiar (−)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)
Negacja U2	-x = ~x+1 ; ~TMin=TMin , -0=0
Wykr. nadmiaru (s=x+y)	((s^x)&(s^y))>>(w-1)
Maska / abs	m=x>>31; abs=(x^m)-m

Pamięć: adres wskazuje najmłodszy bajt słowa. Napis = **char[]** ASCII + **0x00** (niezależny od endianness).

Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

6. Rejestry x86_64

%rax	%eax	%ax %ah %al	%rbx	%ebx	%bx %bh %bl
------	------	----------------	------	------	----------------

%rcx	%ecx	%cx %ch %cl	%rdx	%edx	%dx %dh %dl
------	------	----------------	------	------	----------------

%rsi	%esi	%si %sil	%rdi	%edi	%di %dil
%rbp	%ebp	%bp %bpl	%rsp	%esp	%sp %spl
%r8	%r8d	%r8w %r8b	%r9	%r9d	%r9w %r9b

Uwaga: Rejestry od %r10 do %r15 używają schematu: %r10, %r10d, %r10w, %r10b.

Rejestry ogólnego przeznaczenia

- %rax** Akumulator. Główny rejestr arytmetyczny. Przechowuje **wartość zwracaną** z funkcji.
- %rbx** Rejestr bazowy. Callee-saved.
- %rcx** Licznik w pętlach i przesunięciach bitowych. **4. argument**.
- %rdx** Rejestr danych. Używany w mnożeniu/dzieleniu. **3. argument**.

Indeksy i wskaźniki

- %rdi** Indeks docelowy (DI). **1. argument**.
- %rsi** Indeks źródłowy (SI). **2. argument**.
- %rsp** Wskaźnik stosu (SP). Wskazuje wierzchołek ramki. Modyfikowany przez `push` / `pop` / `call` / `ret`.
- %rbp** Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.

Nowe i specjalne rejestry

- %r8-9** Kolejno: **5. i 6. argument**. Caller-saved.
- %r10 oraz %r11** Rejestry tymczasowe. Caller-saved.
- %r12 do %r15** Ogólnego przeznaczenia. Wszystkie **callee-saved**.
- %rip** Wskaźnik instrukcji (IP). Następną instrukcja. Modyfikowany przez skoki i wywołania.
- %rflags** Rejestr flag statusu. Zmieniany przez `cmp`, `test`.

Rejestry wektorowe (zmiennoprzecinkowe)

- %xmm0 do %xmm15** 128-bitowe rejestry. **%xmm0** to **wartość zwracana** (float/double). **%xmm0-%xmm7** to pierwsze **8 argumentów**. Wszystkie są **caller-saved**.
- %ymm0 do %ymm15** 256-bitowe rozszerzenie rejestrów XMM. Dzielą z nimi najmłodsze 128 bitów.

7. Flagi stanu (Rejestr %rflags)

- ZF** Zero. Wynik to 0 (np. argumenty są równe).
- SF** Sign. Wynik jest ujemny (MSB = 1).
- CF** Carry. Przepelnienie dla liczb **bez znaku** (unsigned).
- OF** Overflow. Przepelnienie dla liczb **ze znakiem** (signed).

8. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Immediate)	\$Imm	Imm
Rejestrowy (Register)	%Ra	%Ra
Bezpośredni (Direct)	Imm	M[Imm]
Pośredni (Indirect)	(%Rb)	M[%Rb]
Z przesunięciem (Indirect displacement)	D(%Rb)	M[%Rb + D]
Skalowany (Indirect scaled-index)	D(%Rb, %Ri, S)	M[%Rb+%Ri*S+D]
Skalowany (baseless)	(, %Ri, S)	M[%Ri * S]

Legenda: Imm to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko wartości: 1, 2, 4 lub 8).

9. Konwencja wywoływań (System V AMD64 ABI)

Przekazywanie argumentów

%rdi → %rsi → %rdx → %rcx → %r8 → %r9

Argumenty 7+ Na stosie od końca.
Wynik Zawsze w %rax.

Ochrona rejestrów (ABI)

Caller-saved
(Można niszczyć)

%rax, %rdi, %rsi, %rdx, %rcx, %r8 - %r11

Callee-saved
(Musi przywrócić)

%rbx, %rbp, %r12 - %r15

Ramka Stosu

Start (alokacja)

```
push %rbp
mov %rsp, %rbp
sub $32, %rsp
```

Koniec (sprzątanie)

```
leave
ret
/* równowaznie:
* mov %rbp, %rsp
* pop %rbp
* ret */
```

10. Struktury i wyrównanie

- Wyrównanie pola (K):** Zmienna K -bajtowa pod adresem $p \bmod K = 0$.
- Rozmiar całkowity:** Całkowity `sizeof` struktury musi być podzielny przez największe wyrównanie w strukturze ($K_{\max}$).

```
struct {
    char c; // 1 bajt (offset 0)
           // [3 bajty wewn. paddingu]
    int i; // 4 bajty (offset 4)
    short s; // 2 bajty (offset 8)
           // [2 bajty zewn. paddingu]
} // K_max = 4. Rozmiar: 1+3+4+2+2 = 12 bajtów.
```

11. Najważniejsze instrukcje (AT&T)

Opcode	Efekt	Opis
Przesyłanie danych		
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos.
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ $\text{pop } \%rbp$	Sprząta ramkę stosu.
Adresowanie i arytmetyka		
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>inc / dec D</code>	$D \leftarrow D \pm 1$	Inkrementacja / dekrementacja.
<code>neg D</code>	$D \leftarrow -D$	Negacja arytmetyczna (jak w U2).
Dzielenie (zawsze w parze)		
<code>cqto</code>	$\%rdx:\%rax \leftarrow \text{sgn_ext}(\%rax)$	Uwaga: Rozszerza $\%rax$ do 128-bit przed <code>idivq</code> .
<code>idiv S</code>	$\%rax \leftarrow \text{wynik}$ $\%rdx \leftarrow \text{reszta}$	Dzielenie $\%rdx:\%rax$ przez S. Wynik w $\%rax$, reszta w $\%rdx$.

Logiczne i bitowe		
<code>and / or S, D</code>	$D \leftarrow D \& \mid S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często używane jako <code>xor %rax, %rax</code> do zerowania.
<code>not D</code>	$D \leftarrow \sim D$	Negacja bitowa (odwrócenie bitów).
<code>sal / shl k, D</code>	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.

Opcode	Efekt	Opis
<code>shr k, D</code>	$D \gg = k$	Logiczne w prawo. Dopelnia zerami.
Porównania i skoki		
<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND (np. test czy rejestr jest zerem).
<code>jX dest</code>	$\text{if}(X) \ \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia bajt (np. <code>%al</code>) na 0 lub 1 na podstawie flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (optymalizacja zamiast skoku).
<code>call / ret</code>		Skok do funkcji / Powrót (obsługa stosu i <code>%rip</code>).

Operacje wektorowe		
<code>movaps / movups S, D</code>	$D \leftarrow S$	Kopiuje 128-bitów. <code>aps</code> (aligned) – wymaga wyrównania w pamięci, <code>ups</code> nie.
<code>movsd / movss S, D</code>	$D \leftarrow S$	Kopiuje pojedynczą wartość skalarną (<code>double</code> / (s) <code>float</code>).
<code>add/sub/mul/div pd/ps</code>	$D \leftarrow D \text{ op } S$	Arytmetyka wektorowa. Wykonuje operację na wielu parach równocześnie.
<code>add/sub/mul/div sd/ss</code>	$D \leftarrow D \text{ op } S$	Arytmetyka skalarna. Modyfikuje tylko najmlodsza wartość w rejestrze.
<code>vxorps S1, S2, D</code>	$D \leftarrow S2 \wedge S1$	Wektorowy XOR .
<code>v... (np. vmulss)</code>	$D \leftarrow S2 \text{ op } S1$	Przedrostek <code>v</code> wprowadza 3 argumenty (źródło 1, źródło 2, cel). Nie niszczy danych wejściowych.
<code>vfmadd231ss S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add. Mnoży i dodaje. Cyfry <code>231</code> określają, że mnożymy dwa źródła, a wynik dodajemy do <code>D</code> .

Uwaga o sufiksach: Instrukcje mogą przyjmować sufiks określający rozmiar danych: `b` (byte, 8-bit), `w` (word, 16-bit), `l` (long, 32-bit), `q` (quadword, 64-bit). Np. `movq` kopiuje 64 bity, a `movd` kopiuje 32 bity (i zeruje górną połowę rejestru!).

12. Sufiksy warunkowe (dla jX , setX , cmovX)

Sufiks	Synonim	Znaczenie / Warunek
Równość i znak (ZF , SF)		
e	z	Equal / Zero (równe / wynik to 0)
ne	nz	Not Equal / Not Zero (nierówne)
s		Sign (wynik ujemny, SF=1)
ns		Not Sign (wynik dodatni lub 0, SF=0)
Liczby ze znakiem (signed)		
g		Greater
ge		Greater or Equal
l		Less
le		Less or Equal
Liczby bez znaku (unsigned)		
a		Above (powyżej: >)
ae	nc	Above or Equal / No Carry (większe równe: >=)
b	c	Below / Carry (poniżej: <)
be		Below or Equal (mniejsze równe: <=)

13. Optymalizacje i atrybuty

Podstawowe techniki	
Strength red.	Zamiana drogich operacji na tańsze (np. mul → lea / shl).
Loop unroll.	Rozwinięcie pętli (np. skok co 4. iterację). Mniej skoków = mniejszy narzut.
Code motion	Wyrzucenie obliczeń niezmienników (np. stałych w pętli) przed pętlę.
Common subexpr. elim.	Obliczenie wspólnego fragmentu tylko raz.
Const folding	Obliczanie stałych wyrażeń (np. 2+2) w czasie kompilacji.
Branch predict	Sprzętowe przewidywanie skoków. By unikać kar, można użyć cmovX .
Inlining	Wklejenie ciała funkcji w miejsce wywołania. Eliminuje narzut call / ret .

Analiza pamięci i atrybuty GCC	
Aliasing	Problem nakładania się wskaźników. Keyword restrict , pomaga w optymalizacji.
pure / const	Funkcja bez skutków ubocznych (const dodatkowo nie czyta zmiennych globalnych).

14. Bezpieczeństwo kodu i exploity

Zagrożenia i błędy	
Buffer overflow	Zapis poza limit bufora. Nadpisuje sąsiadującą pamięć.
Seg fault	Próba dostępu bez uprawnień (np. odczyt NULL , modyfikacja read-only).

Stack smashing	Atak nadpisujący adres powrotu na stosie, by przejąć kontrolę po ret .
ROP (Gadżety)	Łączenie legalnych instrukcji kończących się ret w celu ominięcia zabezpieczeń.
Mechanizmy obronne	
Stack canaries	Losowa wartość ułożona przed adresem powrotu. Weryfikowana przed ret .
Nonexec code segments	Stos dostaje sprzętowy zakaz wykonywania kodu (tylko R/W).
Rand. stack offsets	Losowanie adresów stosu przy każdym starcie.